

Быстрый вкат в языковые фреймворки

Архитектура LLM систем, основные инструменты, некоторые best-practice паттерны, “сеньорское” мышление

Что такое LLM система?

- Основная идея: LLM сама по себе не всегда даёт полезный результат. Чтобы улучшить результат, нужна архитектура, инструменты, данные.
- Можно описать в виде нескольких основных слоев: слой модели, слой промптов, слой оркестрации, слой данных, слой observability.
- Использует языковые фреймворки: LangChain, LangGraph, Langfuse, RAG системы или другие

Ядро LLM системы

- Самый популярный вариант реализации - Langchain.
- Можно отнести к слою оркестрации, является центральным “мозгом” координирующим действия между другими слоями
- Основная ценность Langchain - добавляет набор понятных абстракций (PromptTemplate, Tool, Agent, LLM), позволяющих быстро выстраивать систему



Синтаксис Langchain

```
from langchain.chat_models import init_chat_model

model = init_chat_model(
    model: "openai:gpt-4o",
    temperature=.7,
    api_key=api_key,
)

resp = model.invoke("explain what rag means?")

if __name__ == '__main__':
    print(resp)
```

Синтаксис Langchain

```
from langchain.chat_models import init_chat_model

model = init_chat_model(
    model="openai:gpt-4o",
    temperature=.7,
    api_key=api_key,
)

if __name__ == '__main__':
    for chunk in model.stream("Give a step-by-step explanation of RAG"):
        print(chunk.text, end="")
```

Синтаксис Langchain

```
from langchain.chat_models import init_chat_model

model = init_chat_model(
    model: "openai:gpt-4o",
    temperature=.7,
    api_key=api_key,
)

if __name__ == '__main__':
    results = model.batch([
        "What is LangChain?",
        "What is LangGraph?"
    ])
    for r in results:
        print(r)
```


Синтаксис Langchain

- Цепочка действий: User Prompt → Prompt Template → RAG → LLM → Output

Основные абстракции в Langchain

- LLMs
- Chains
- Memory
- Agents
- Tools

LLM-агент

- LLM модель в системе, направленная на работу над одной отдельной глобальной задачей;
- Имеет возможность вызова инструмента из списка доступных инструментов;
- Выбирает инструмент отталкиваясь от системного промпта, промпта пользователя, сигнатуры инструмента;

LLM-agent

```
from langchain.agents import create_agent

def calc_profit(report: str) -> str:
    """Get profit from the report."""
    profit: int = 100
    return f"Profit for this report is {profit}"

agent = create_agent(
    model="claude-sonnet-4-6",
    tools=[calc_profit],
    system_prompt="You are an economic assistant",
)

agent.invoke(
    {"messages": [{"role": "user", "content": "What is the profit in this report: ..."}]}
```

LLM-agent

```
from langchain.agents import create_agent ⚠️ 1 ⚠️ 2

def calc_profit(report: str) -> str: # usage
    """Get profit from the report."""
    profit: int = 100
    return f"Profit for this report is {profit}"

def forecast(data: list) -> str: # usage
    """Give forecast based on data ranges provided."""
    ...

    return "Forecast is ..."

agent = create_agent(
    model="claude-sonnet-4-6",
    tools=[calc_profit, forecast],
    system_prompt="You are an economic assistant",
)

agent.invoke(
    {"messages": [{"role": "user", "content": "What is the profit in this report: ..."}]}
```


Агентная система

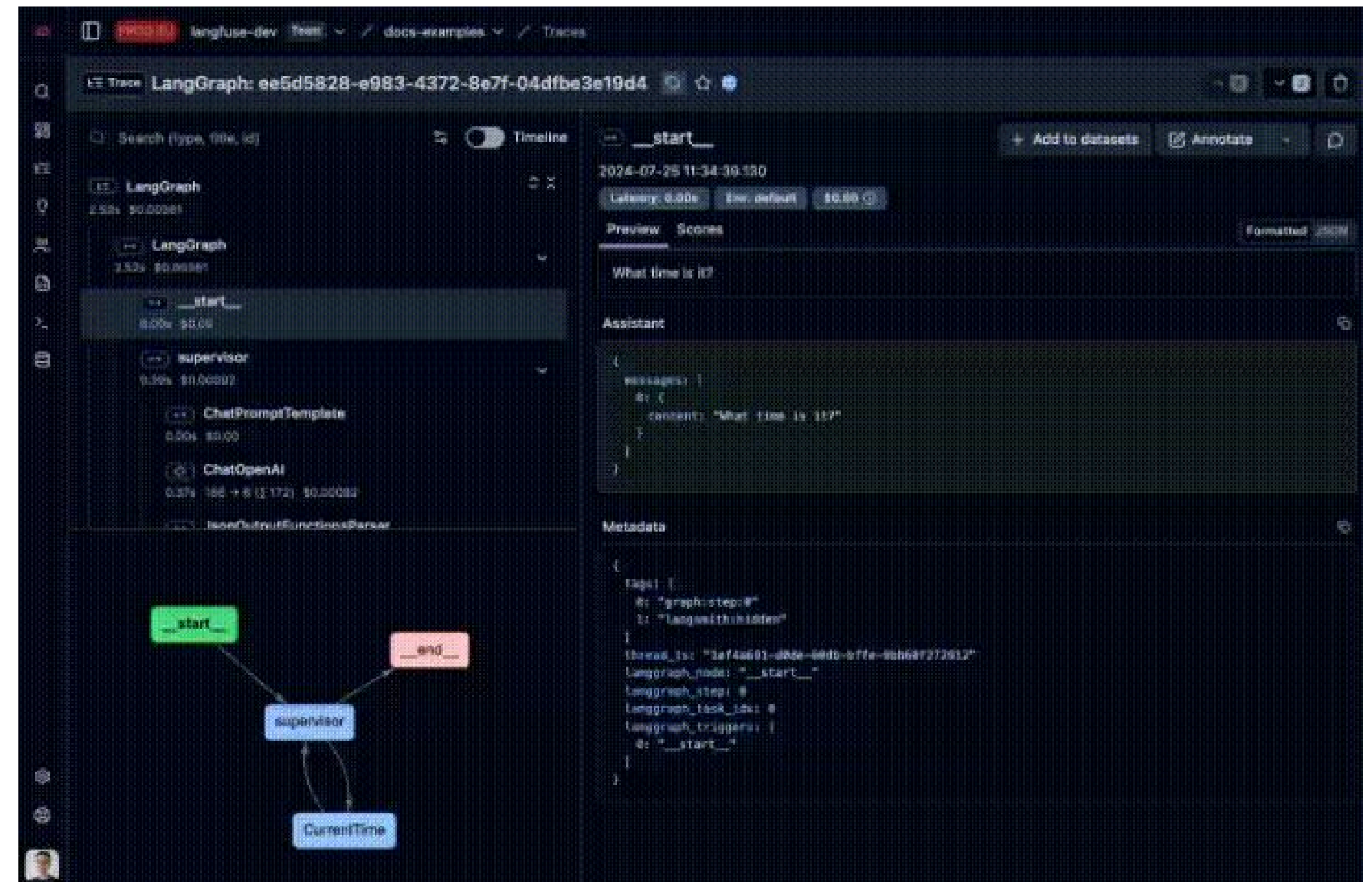
- Строится на взаимодействии нескольких агентов;
- Каждый агент глобально закрывает свой отдельный домен (финансовый агент, агент получения общих данных, агент данных по внутреннему производству).
- Агенты могут передавать задачи друг другу, иметь контекстную память.
- Помимо своих собственных тулов, каждый агент может иметь свой собственный промпт который может “направить” модель в правильное направление.
- Также каждый агент может быть построены на основе fine-tuned модели

Langgraph

- Langgraph - надстройка над Langchain, позволяющая ввести парадигму графов в систему LLM
- Каждая операция становится отдельным узлом
- Введение условий в системе, retry логика
- StatefulGraph

Observability: Langfuse

- Встроенная open-source платформа для трейсинга
- Позволяет отслеживать цепочки вызовов, выбор узлов в графах, контекст агентов, результаты работы тулов
- Удобный UI
- Мета данные



Другие ключевые концепции

- RAG: Retrieval → Augmentation → Generation
- Эмбединг - числовое представление текста
- Чанкинг - разбиение текста на фрагменты
- Галлюцинации модели - генерация правдоподобного, но неверного контента моделью

Вопросы которые могут задать

- Агент вызывает неверный тул, или не вызывает тул вообще. В чем может быть потенциальная проблема и как будешь дебажить?
- Что такое агент? Зачем его интегрировать в систему?
- Что такое мультиагентная система? Каковы потенциальные слабости мультиагентной системы?
- Для чего нужны эмбединги в LLM системе?
- Для чего в системах используется RAG?
- Может ли RAG помочь в борьбе с галлюцинациями?

Вопросы которые могут задать

- Зачем нужен LangChain, если можно написать всё на Python?
- В чём разница между Chain, Agent и LangGraph? Когда что использовать?
- Как интегрировать большие документы в LLM систему максимально эффективно?
- Что лучше для production: fine-tuning или RAG? Почему?
- Что такое multi-head RAG? В чем его преимущество?
- Какие есть основные абстракции используемые в Langchain?
- Что такое StatefulGraph?